Zodiac

A real-hardware testbed and verifier for agent-proposed SDN policy

Document: Position paper with a measured foundation
Version: 2.0 (draft) **Date:** 2026-10-09
Status: Measurements final (2026-10-04). Verifier partly implemented. Evaluation planned, no results reported.
Repository: github.com/gravity-tpu/agentic-era-networking

Abstract. Agents can generate network policy faster than people can review it, so the scarce resource is verification. We describe Zodiac, a testbed for studying that problem on physical hardware: an OpenFlow 1.3 switch with a 128-rule table, a mirror tap, an instrumented controller, and a hostile workload node. We report the measured hardware model. Forwarding cost is `38.5 µs + 34.8 ns per payload byte above 64 + 4.24 µs per rule`, set by the switch's microcontroller and not its 100 Mbit/s ports; rule layout changes capacity by up to 2.6×; and offered load of 1.3 to 2.3 times capacity makes the switch drop its control session and its flow table. We argue that the right verifier for this system checks properties of the rule set, including capacity, against that measured model, and closes the loop by testing model against wire. The verifier is partly built; the evaluation, including how often LLM-proposed rule sets are unsafe, is specified here and not yet run.

1. Problem

An agent asked to "let the workload reach DNS but nothing else" can produce a plausible flow-rule set in seconds. Whether it is safe is a different question, and the cost asymmetry between producing and checking policy grows as generation gets cheaper. Review by a person does not scale, and review by another model is a second opinion, not a proof.

Most work on checking network policy runs on configuration snapshots or in emulation. Those settings omit what a deployment actually fails on: a small switch whose CPU is the bottleneck, firmware that supports a subset of the match fields, and control sessions that collapse under load. Zodiac's contribution is to ground the question in a real device whose limits are measured, so a policy can fail the verifier for a hardware reason (too many rules, wrong layout, too much load) as well as a logical one (isolation leak).

The question we study: *can an agent safely propose network policy, and what must "safely" mean on hardware this constrained?* We do not claim autonomous operation. Agents propose; deterministic code decides admissibility; a person approves.

2. Testbed

Role	Device
Device under test	Northbound Networks Zodiac FX, firmware 0.86: SAM4E8E 120 MHz MCU, KSZ8795 switch IC, 4 x 100BASE-TX, 128-rule flow table, OpenFlow 1.3
Controller, benchmark host, detector	NVIDIA Jetson Orin Nano Super (two Coral Edge TPUs for inference)
Hostile workload, traffic generator	Raspberry Pi 5 with Hailo-8; Raspberry Pi 3B+ as the client workload
Workload aggregation	NETGEAR GS105Ev2 (802.1Q, port mirror)

Port 1 carries the untrusted workload segment, Port 2 the uplink to the lab LAN and Internet, Port 3 a mirror tap read by the Jetson, and Port 4 the OpenFlow control channel. The policy under test isolates the workload: only IPv4 whose source and destination are both inside the workload subnet, plus ARP, may cross Port 1 to Port 2; everything else from Port 1 is mirrored to the tap only; the table-miss entry drops.

Provenance. Every measurement artifact is stamped as a physical run, uses 64-bit integer counters and monotonic nanosecond clocks, and binds the switch identity and firmware version. Telemetry from disconnected hardware reports zero and offline, never a plausible value. Modelled values in this paper are marked as modelled.

3. The measured hardware model

Traffic from the Pi to a receiver on the uplink was offered at fixed rates in 45 s steps per rule depth, and forwarded rate was read from the switch's own port counters. Full method, data and limitations are in the benchmark whitepaper [1].

```
t_packet = 38.5 µs + 0.0348 µs x (B − 64) + 4.24 µs x N      (base layout; B = UDP
payload bytes, N = rules)
capacity = 1,000,000 / t_packet  packets per second
```

The equation was fitted on the 64-byte and 572-byte ladders and predicted the independent 1470-byte ladder within about 1% at three rule depths. The switch evaluates its table in software, so a packet is limited by the MCU and the number of rules scanned before its match, not by the 100 Mbit/s port. Port speed only binds for large packets with almost no rules.

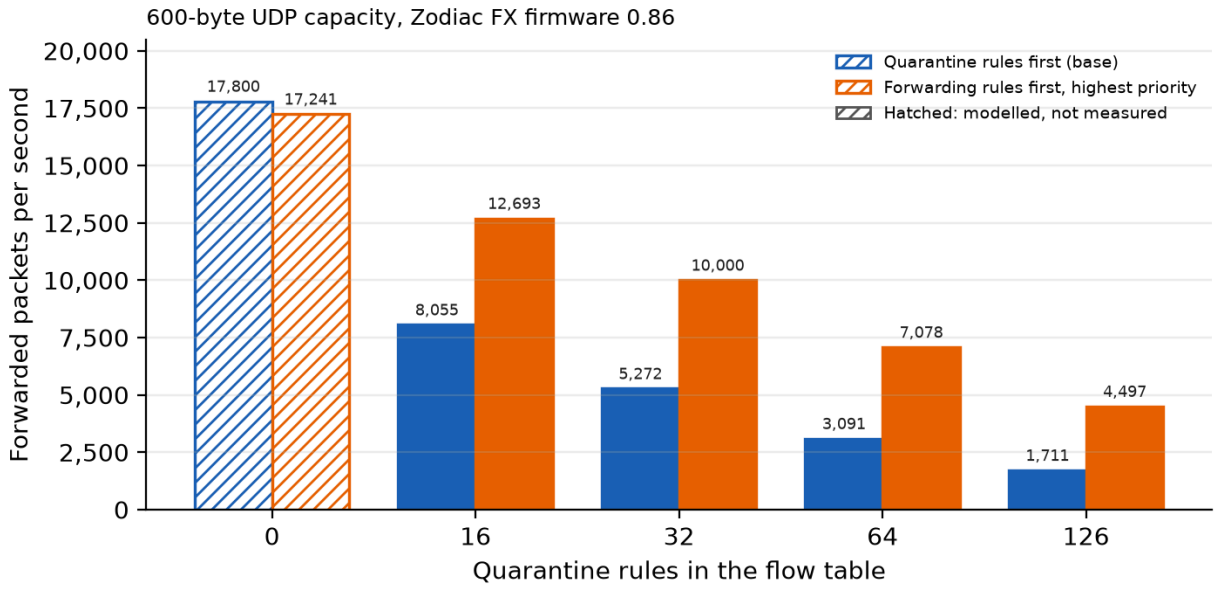


Figure 1. Forwarded packets per second for 600-byte UDP at each quarantine rule depth, for two table layouts. Solid bars are measured; hatched bars are modelled from the fitted equation (empty table: the base run did not saturate, at least 14,807 pps measured, and the forwarding-first run lost its session). Values from [1], section 5.1. Single rounds for most layouts.

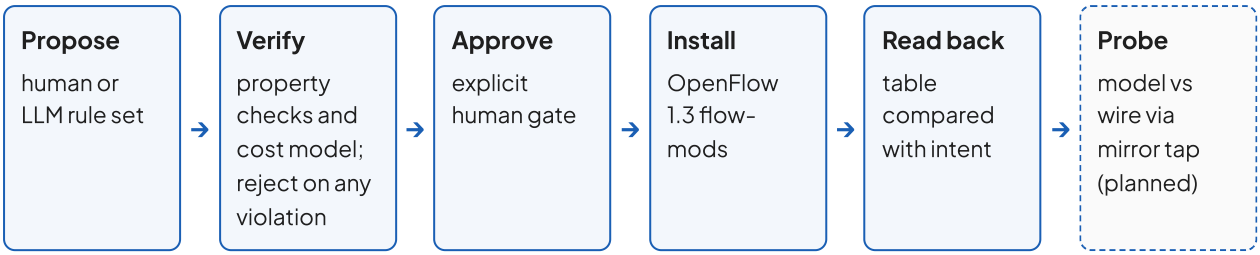
Findings that constrain any verifier

- **Layout is a first-order effect.** Installing forwarding rules first and at the highest priority cuts the per-rule cost from 4.24 µs to 1.3 µs, which gives 1.6× to 2.6× more capacity at 16 to 126 rules. Either change alone did not help.
- **The control plane is the fragile part.** At 1.3 to 2.3 times capacity the switch closes its OpenFlow session and clears its table; recovery takes about 2 to 3 s, and under sustained overload the table could not be restored. Spontaneous session losses also occurred below capacity (8 in three working-window hours).
- **Availability under a realistic working pattern** (rules accumulating over five minutes, then flush) was 99.83% to 99.95% over three one-hour runs.
- **The original 20,800 pps plan is unreachable.** At 600 bytes the empty-table ceiling is about 17,800 pps.

Consequence: a policy that is logically correct can still make the switch unusable. Capacity and layout therefore belong in the set of properties a verifier checks.

4. Approach: verify the rule set against the model

A static flow table has no routing loops to find, so a loop-freedom check on hop lists proves little here. (An earlier version of this project used one and called it a symbolic verifier; its call sites passed one-hop lists that cannot fail. We treat it as superseded.) The useful questions are properties of the rule set and its interaction with the hardware.



Id	Property	State
A1	Isolation: nothing from the workload port reaches the uplink except explicitly allowed traffic	Implemented
A2	No hairpin: no rule outputs to its own ingress port	Implemented
A3	Default drop: a lowest-priority table-miss DROP exists	Readback only
A4	Quarantine precedence over every allow rule	By constant; check planned; conflicts with A9
A5, A6	No shadowed rules; no conflicting overlaps at equal priority	Planned
A7	Rule budget within the 128-rule table, with runtime headroom	Planned
A8	Modelled capacity at this rule count and layout meets a declared target rate	Planned (uses the model in section 3)
A9	Layout: forwarding rules ordered per the measured cheap layout	Planned; conflicts with A4
A10	ARP confinement across the isolation boundary	Blocked: firmware 0.86 ARP field matching unverified

Today the implementation checks A1 and A2 statically (`verifyPort1Isolation`) and compares the switch's flow-stats readback with intent on match fields (`verifyIsolationReadback`). Readback does not yet parse output actions, so it proves what the switch matches, not where it forwards. Closing that gap, and the planned checks, are the near-term work.

A tension between cost and containment

The cheapest measured layout (forwarding rules first, at the highest priority) puts forwarding rules above quarantine rules. A quarantined source that also matches a forwarding rule would still be forwarded, so properties A4 and A9 cannot both hold as written. The benchmark is unaffected because its quarantine rules match addresses that never appear on the wire. Resolving this, by excluding quarantined sources from forwarding rules, splitting quarantine rules across priorities, or accepting the quarantine-first cost, is the first design question for the verifier, and the capacity cost of each option should be measured.

A known weakness in the current policy

The isolation policy forwards ARP between the workload and the lab LAN, because firmware 0.86 support for ARP source and target fields is unverified. The code documents this as address disclosure only. We suspect it is also a spoofing surface: a workload can send ARP claiming a lab-

LAN address. This is a hypothesis; testing it is the first adversarial scenario below, and a confirmed result becomes a verifier property.

5. Evaluation plan

None of the following has been run. It is specified so the results are interpretable, and so a negative result is reportable.

E1. Model versus wire (conformance)

For a stated set of generated packets, compute the model's verdict, send the packets from the workload node, and observe what leaves each port through the mirror tap. Metric: agreement rate, with each disagreement classified as a model error, a firmware behavior, or a harness artifact. Falsifier: systematic disagreement on forwarding or drop decisions means the static checks cannot be trusted for this firmware.

E2. Adversarial scenarios from the Pi 5

1. ARP spoofing across the isolation boundary.
2. MAC and IP spoofing within the workload segment.
3. Port scan and low-rate scan.
4. Flood to session loss (what an attacker can do to the control plane).
5. Evasion: traffic crafted to fit allowed rules, fragmentation, rule-table exhaustion.

Metrics per scenario: contained or not, time to detect, time to quarantine, false positives, and whether the verifier or the detector would have prevented it. The existing anomaly detector (Coral TPU and Jetson GPU) is the detector under test; its precision and recall are unmeasured today.

E3. Agent-proposal evaluation

For a fixed set of natural-language intents, have one or more LLMs propose rule sets. Metrics: fraction rejected by the verifier and by which property; fraction accepted that misbehave on hardware (false accepts, the dangerous case); fraction rejected that were in fact safe (false rejects). Baselines: unverified install, and the same intents written by a person. This is the project's central empirical question and depends on E1 and a sufficiently rich property set.

6. Related work

Static data-plane verification is established. Header Space Analysis [3] and VeriFlow [4] check reachability and loop properties over forwarding state in software, and Batfish [5] analyzes network configurations. Zodiac does not claim a new verification algorithm. It differs by (i) pairing the verifier with a measured model of one commodity device, so capacity and layout are checked properties; (ii) comparing the model against traffic on the wire; and (iii) evaluating agent-generated policy specifically. We have not surveyed the literature exhaustively or compared against these tools.

7. Limitations and threats to validity

- **One switch, one firmware, one workload node.** The cost model is fitted to a single Zodiac FX on firmware 0.86 and must not be assumed to hold elsewhere. Several layout ladders are single rounds.
- **Open measurements:** drop-path capacity (the drop-rule hit counter stayed at zero while traffic was dropped), direct observation of switch CPU load, and unexplained spontaneous session losses.
- **The verifier encodes the authors' reading of L2 and flow-table semantics.** It has had no independent network-engineer review; that is planned before results are published.
- **The anomaly detector's accuracy is unmeasured.** Claims about detection are deferred to E2.
- **LLM nondeterminism** affects E3; it requires repeated runs and reporting variance.
- **Scale.** This is a lab on a 100 Mbit/s MCU switch. It makes no availability or production-readiness claim, and results do not extrapolate to ASIC switches.

8. Status

Item	State
Hardware model and benchmark harness	Measured, documented [1]
Isolation policy and static checks A1, A2; readback on match fields	Implemented and unit-tested
Checks A3 to A10; action-level readback; cost-model input to the verifier	Planned
E1 conformance, E2 adversarial suite, E3 agent evaluation	Planned; no results
Independent review of policy and property list	Planned before publication of results

The roadmap [2] gives sequencing and exit criteria. Deferred or dropped from the earlier plan: the Sentinel bot-mitigation module, DNS reverse proxy, multi-agent routing stability, and the agent intent firewall.

References

1. Zodiac Network Infrastructure benchmark archive, *Packet-Rate Limits of an OpenFlow Edge Switch Under Rule-Table Load*, v1.1, 2026. `docs/benchmarks/zodiac-infrastructure-bandwidth-whitepaper-2026-10-04.md`
2. Zodiac roadmap. `docs/ROADMAP.md`
3. P. Kazemian, G. Varghese, N. McKeown, "Header Space Analysis: Static Checking for Networks," NSDI 2012.
4. A. Khurshid, X. Zou, W. Zhou, M. Caesar, P. B. Godfrey, "VeriFlow: Verifying Network-Wide Invariants in Real Time," NSDI 2013.
5. A. Fogel et al., "A General Approach to Network Configuration Analysis," NSDI 2015.
6. Open Networking Foundation, *OpenFlow Switch Specification, Version 1.3.5*, 2015.